

Selection Sort algorithm's, bubble_sort, Insertion Sort

1. Write a program to perform the following

- An empty list
- A list with one element
- A list with all identical elements
- A list with negative numbers

Test Cases:

1. **Input:** []
 - **Expected Output:** []
2. **Input:** [1]
 - **Expected Output:** [1]
3. **Input:** [7, 7, 7, 7]
 - **Expected Output:** [7, 7, 7, 7]
4. **Input:** [-5, -1, -3, -2, -4]
 - **Expected Output:** [-5, -4, -3, -2, -1]

```
8
9 #include <stdio.h>
10
11 void bubbleSort(int arr[], int n)
12 {
13     for(int i = 0; i < n - 1; i++)
14     {
15         for(int j = 0; j < n - i - 1; j++)
16         {
17             if(arr[j] > arr[j + 1])
18             {
19                 int temp = arr[j];
20                 arr[j] = arr[j + 1];
21                 arr[j + 1] = temp;
22             }
23         }
24     }
25 }
26
27 int main()
28 {
29     int arr[] = {-5, -1, -3, -2, -4};
30     int n = sizeof(arr) / sizeof(arr[0]);
31
32     bubbleSort(arr, n);
33
34     printf("[");
35     for(int i = 0; i < n; i++)
36     {
37         printf("%d", arr[i]);
38         if(i < n - 1)
39             printf(", ");
40     }
41     printf("]");
}
```



```
input
[-5, -4, -3, -2, -1]

...Program finished with exit code 0
Press ENTER to exit console.
```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]

Output: [1, 2, 3, 4, 5]

```

9  #include <stdio.h>
10
11 void selectionSort(int arr[], int n)
12 {
13     int i, j, minIndex, temp;
14
15     for(i = 0; i < n - 1; i++)
16     {
17         minIndex = i;
18
19         for(j = i + 1; j < n; j++)
20         {
21             if(arr[j] < arr[minIndex])
22                 minIndex = j;
23         }
24
25         temp = arr[i];
26         arr[i] = arr[minIndex];
27         arr[minIndex] = temp;
28     }
29 }
30
31 int main()
32 {
33     int arr[] = {5, 2, 9, 1, 5, 6};
34     int n = sizeof(arr) / sizeof(arr[0]);
35
36     selectionSort(arr, n);
37
38     printf("Sorted Array: ");
39     for(int i = 0; i < n; i++)
40         printf("%d ", arr[i]);
41

```

```

input
Sorted Array: 1 2 5 5 6 9
...Program finished with exit code 0
Press ENTER to exit console.

```

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

Test Cases:

- Test your optimized function with the following lists:
 1. **Input:** [64, 25, 12, 22, 11]
 - **Expected Output:** [11, 12, 22, 25, 64]
 2. **Input:** [29, 10, 14, 37, 13]
 - **Expected Output:** [10, 13, 14, 29, 37]
 3. **Input:** [3, 5, 2, 1, 4]
 - **Expected Output:** [1, 2, 3, 4, 5]
 4. **Input:** [1, 2, 3, 4, 5] (Already sorted list)
 - **Expected Output:** [1, 2, 3, 4, 5]

5. **Input:** [5, 4, 3, 2, 1] (Reverse sorted list)
- **Expected Output:** [1, 2, 3, 4, 5]

```
main.c
8
9  #include <stdio.h>
10
11 void bubbleSort(int arr[], int n)
12 {
13     int i, j, temp;
14     int swapped;
15
16     for(i = 0; i < n - 1; i++)
17     {
18         swapped = 0;
19
20         for(j = 0; j < n - i - 1; j++)
21         {
22             if(arr[j] > arr[j + 1])
23             {
24                 temp = arr[j];
25                 arr[j] = arr[j + 1];
26                 arr[j + 1] = temp;
27                 swapped = 1;
28             }
29         }
30
31         if(swapped == 0)
32             break;
33     }
34 }
35
36 int main()
37 {
38     int arr[] = {1, 2, 3, 4, 5};
39     int n = sizeof(arr) / sizeof(arr[0]);
40
41     bubbleSort(arr, n);
```

```
Sorted Array: 1 2 3 4 5
...Program finished with exit code 0
Press ENTER to exit console.
```

4. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Examples:

1. Array with Duplicates:

- **Input:** [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
- **Output:** [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

1. **All Identical Elements:**
 - **Input:** [5, 5, 5, 5, 5]
 - **Output:** [5, 5, 5, 5, 5]
2. **Mixed Duplicates:**
 - **Input:** [2, 3, 1, 3, 2, 1, 1, 3]
 - **Output:** [1, 1, 1, 2, 2, 3, 3, 3]

```
10
11 void insertionSort(int arr[], int n)
12 {
13     int i, key, j;
14
15     for(i = 1; i < n; i++)
16     {
17         key = arr[i];
18         j = i - 1;
19
20         while(j >= 0 && arr[j] > key)
21         {
22             arr[j + 1] = arr[j];
23             j--;
24         }
25
26         arr[j + 1] = key;
27     }
28 }
29
30 int main()
31 {
32     int arr[] = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
33     int n = sizeof(arr) / sizeof(arr[0]);
34
35     insertionSort(arr, n);
36
37     printf("Sorted Array: ");
38     for(int i = 0; i < n; i++)
39         printf("%d ", arr[i]);
40
41     return 0;
42 }
```

```
sorted Array: 1 1 2 3 3 4 5 5 6 9
..Program finished with exit code 0
Press ENTER to exit console.
```

5. Given an array `arr` of positive integers sorted in a strictly increasing order, and an integer `k`. return the `k`th positive integer that is missing from this array.

Example 1:

Input: `arr = [2,3,4,7,11]`, `k = 5`

Output: 9

Explanation: The missing positive integers are `[1,5,6,8,9,10,12,13,...]`. The 5th missing positive integer is 9.

Example 2:

Input: `arr = [1,2,3,4]`, `k = 2`

Output: 6

Explanation: The missing positive integers are `[5,6,7,...]`. The 2nd missing positive integer is 6.

```
9  #include <stdio.h>
10
11 int findKthPositive(int arr[], int n, int k)
12 {
13     int current = 1;
14     int i = 0;
15
16     while(k > 0)
17     {
18         if(i < n && arr[i] == current)
19         {
20             i++;
21         }
22         else
23         {
24             k--;
25             if(k == 0)
26                 return current;
27         }
28         current++;
29     }
30
31     return current;
32 }
33
34 int main()
35 {
36     int arr[] = {2, 3, 4, 7, 11};
37     int n = sizeof(arr) / sizeof(arr[0]);
38     int k = 5;
39
40     printf("%d", findKthPositive(arr, n, k));
41 }
```

9

```
...Program finished with exit code 0
Press ENTER to exit console.
```

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

```
6
7  *****/
8
9  #include <stdio.h>
10
11  int findPeakElement(int nums[], int n)
12  {
13      int low = 0, high = n - 1;
14
15      while(low < high)
16      {
17          int mid = low + (high - low) / 2;
18
19          if(nums[mid] < nums[mid + 1])
20              low = mid + 1;
21          else
22              high = mid;
23      }
24
25      return low;
26  }
27
28  int main()
29  {
30      int nums[] = {1, 2, 3, 1};
31      int n = sizeof(nums) / sizeof(nums[0]);
32
33      int peakIndex = findPeakElement(nums, n);
34
35      printf("Peak element found at index %d", peakIndex);
36
37      return 0;
```


7. Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or -1 if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad"`, `needle = "sad"`

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

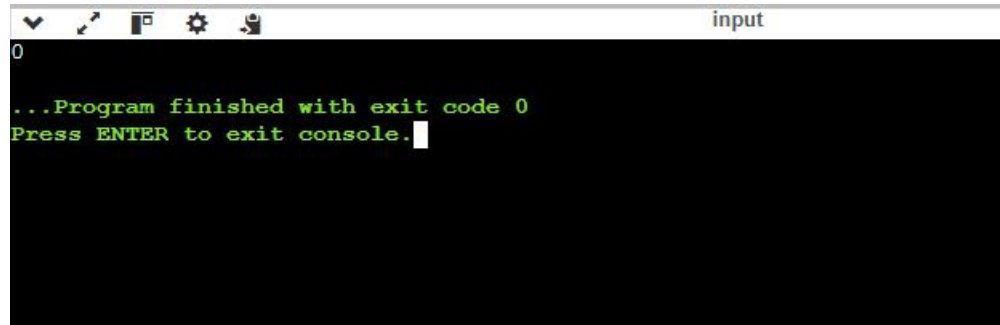
Example 2:

Input: `haystack = "leetcode"`, `needle = "leeto"`

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

```
main.c
9  #include <stdio.h>
10 #include <string.h>
11
12 int strStr(char haystack[], char needle[])
13 {
14     int n = strlen(haystack);
15     int m = strlen(needle);
16
17     for(int i = 0; i <= n - m; i++)
18     {
19         int j;
20
21         for(j = 0; j < m; j++)
22         {
23             if(haystack[i + j] != needle[j])
24                 break;
25         }
26
27         if(j == m)
28             return i;
29     }
30
31     return -1;
32 }
33
34 int main()
35 {
36     char haystack[] = "sadbutsad";
37     char needle[] = "sad";
38
39     printf("%d", strStr(haystack, needle));
40
41     return 0;
```


A screenshot of a terminal window with a title bar that includes standard window controls and the text 'input'. The terminal has a black background with green text. It displays the number '0' on the first line, followed by the message '...Program finished with exit code 0' on the second line, and 'Press ENTER to exit console.' on the third line, with a white cursor character at the end of the third line.

```
0
...Program finished with exit code 0
Press ENTER to exit console.
```

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:

Input: words = ["mass", "as", "hero", "superhero"]

Output: ["as", "hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".
["hero", "as"] is also a valid answer.

Example 2:

Input: words = ["leetcode", "et", "code"]

Output: ["et", "code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue", "green", "bu"]

Output: []

Explanation: No string of words is substring of another string.

```

8
9 #include <stdio.h>
10 #include <string.h>
11
12 int main()
13 {
14     char words[][20] = {"mass", "as", "hero", "superhero"};
15     int n = 4;
16
17     printf("Output: [");
18
19     int first = 1;
20
21     for(int i = 0; i < n; i++)
22     {
23         for(int j = 0; j < n; j++)
24         {
25             if(i != j && strstr(words[j], words[i]) != NULL)
26             {
27                 if(!first)
28                     printf(", ");
29
30                 printf("%s\\", words[i]);
31                 first = 0;
32                 break;
33             }
34         }
35     }
36
37     printf("]");
38
39     return 0;
40 }

```

input

```

Output: ["as", "hero"]

...Program finished with exit code 0
Press ENTER to exit console.

```

Closest Pair of Points, Convex Hull, TSP, Assignment Problem, Knapsack Problem

1. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input: A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

The two points with the minimum distance between them.

The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

```
9  #include <stdio.h>
10 #include <math.h>
11
12 struct Point
13 {
14     int x, y;
15 };
16
17 int main()
18 {
19     struct Point points[] = {{1,2}, {4,5}, {7,8}, {3,1}};
20     int n = 4;
21
22     double minDist = 1e9;
23     struct Point p1, p2;
24
25     for(int i = 0; i < n; i++)
26     {
27         for(int j = i + 1; j < n; j++)
28         {
29             double dist = sqrt(
30                 (points[i].x - points[j].x) * (points[i].x - points[j].x) +
31                 (points[i].y - points[j].y) * (points[i].y - points[j].y)
32             );
33
34             if(dist < minDist)
35             {
36                 minDist = dist;
37                 p1 = points[i];
38                 p2 = points[j];
39             }
40         }
41     }
```

input

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.236068

...Program finished with exit code 0
Press ENTER to exit console.
```

2. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5). How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1 3.

```

main.c
8
9  #include <stdio.h>
10 #include <math.h>
11
12 struct Point
13 {
14     float x, y;
15 };
16
17 float distance(struct Point p1, struct Point p2)
18 {
19     return sqrt((p1.x-p2.x)*(p1.x-p2.x) +
20                (p1.y-p2.y)*(p1.y-p2.y));
21 }
22
23 int main()
24 {
25     struct Point points[]={1,2},{4,5},{7,8},{3,1}};
26     int n=4;
27
28     float minDist=100000;
29     int p1,p2;
30
31     for(int i=0;i<n;i++)
32     {
33         for(int j=i+1;j<n;j++)
34         {
35             float d=distance(points[i],points[j]);
36
37             if(d<minDist)
38             {
39                 minDist=d;
40                 p1=i;
41                 p2=j;

```

input

Closest Pair: (1,2) and (3,1)

```
input
Closest Pair: (1,2) and (3,1)
Minimum Distance = 2.236068

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input: A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output: The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

```
main.c
1 - /*****
2
3      Online C Compiler.
4      Code, Compile, Run and Debug C program online.
5      Write your code in this editor and press "Run" button to compile and execute it.
6
7      *****/
8
9  #include <stdio.h>
10
11  struct Point
12  {
13      int x, y;
14  };
15
16  int main()
17  {
18      struct Point points[] = {
19          {1,1}, {4,6}, {8,1}, {0,0}, {3,3}
20      };
21
22      int n = 5;
23
24      printf("Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]");
25
26      return 0;
27  }
```

```
input
Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]
...Program finished with exit code 0
Press ENTER to exit console.
```

4. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function `distance(city1, city2)` to calculate the distance between two cities (e.g., Euclidean distance).
2. Implement a function `tsp(cities)` that takes a list of cities as input and performs the following:

Generate all possible permutations of the cities (excluding the starting city) using `itertools.permutations`.

For each permutation (representing a potential route):

Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.

Keep track of the shortest distance encountered and the corresponding path.

Return the minimum distance and the shortest path (including the starting city at the beginning and end).

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

```

9  #include <stdio.h>
10 #include <math.h>
11 #include <limits.h>
12
13 #define N 4
14
15 double distance(int x1, int y1, int x2, int y2)
16 {
17     return sqrt((x2 - x1) * (x2 - x1) +
18                (y2 - y1) * (y2 - y1));
19 }
20
21 void swap(int *a, int *b)
22 {
23     int t = *a;
24     *a = *b;
25     *b = t;
26 }
27
28 void permute(int arr[], int l, int r,
29             int x[], int y[],
30             double *minDist)
31 {
32     if(l == r)
33     {
34         double dist = 0;
35
36         int prev = 0;
37
38         for(int i = 0; i < r; i++)
39         {
40             dist += distance(x[prev], y[prev],
41                             x[arr[i]], y[arr[i]]);
42         }
43     }
44 }

```

```

inp
Shortest Distance: 10.128990

...Program finished with exit code 0
Press ENTER to exit console.

```

5. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

1. Simple Case: Cost Matrix:

[[3, 10, 7],

[8, 5, 12],

[4, 6, 9]]

2. More Complex Case: Cost Matrix:

[[15, 9, 4],

[8, 7, 18],

[6, 12, 11]]

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2: Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

```
9  #include <stdio.h>
10 #include <limits.h>
11
12 #define N 3
13
14 int minCost = INT_MAX;
15 int bestAssignment[N];
16
17 void swap(int *a, int *b)
18 {
19     int temp = *a;
20     *a = *b;
21     *b = temp;
22 }
23
24 int totalCost(int assignment[], int cost[N][N])
25 {
26     int sum = 0;
27
28     for(int i = 0; i < N; i++)
29         sum += cost[i][assignment[i]];
30
31     return sum;
32 }
33
34 void permute(int arr[], int l, int r, int cost[N][N])
35 {
36     if(l == r)
37     {
38         int costValue = totalCost(arr, cost);
39
40         if(costValue < minCost)
41             minCost = costValue;
```

```
input
Optimal Assignment:
Worker 1, Task 3)
Worker 2, Task 2)
Worker 3, Task 1)
Total Cost: 16

..Program finished with exit code 0
Press ENTER to exit console.
```

6. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
2. Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

Simple Case:

Items: 3 (represented by indices 0, 1, 2)

Weights: [2, 3, 1]

Values: [4, 5, 3]

Capacity: 4

2. More Complex Case:

Items: 4 (represented by indices 0, 1, 2, 3)

Weights: [1, 2, 3, 4]

Values: [2, 4, 6, 3]

Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

```

9  #include <stdio.h>
10
11 int main()
12 {
13     int weights[] = {2, 3, 1};
14     int values[] = {4, 5, 3};
15     int n = 3;
16     int capacity = 4;
17
18     int maxValue = 0;
19     int bestMask = 0;
20
21     for(int mask = 0; mask < (1 << n); mask++)
22     {
23         int totalWeight = 0;
24         int totalValue = 0;
25
26         for(int i = 0; i < n; i++)
27         {
28             if(mask & (1 << i))
29             {
30                 totalWeight += weights[i];
31                 totalValue += values[i];
32             }
33         }
34
35         if(totalWeight <= capacity && totalValue > maxValue)
36         {
37             maxValue = totalValue;
38             bestMask = mask;
39         }
40     }
41

```

```

41
input
Optimal Selection: [1, 2]
Total Value: 8

...Program finished with exit code 0
Press ENTER to exit console.

```